

# Apprendimento Supervisionato per la Classificazione

Metodi Statistici e Matematici per il Machine Learning

Traduzione italiana — materiale originale a cura di Davide Evangelista

Nei capitoli precedenti abbiamo introdotto la pipeline generale del Machine Learning e sviluppato una comprensione dettagliata delle tecniche di ottimizzazione come la Discesa del Gradiente e la Discesa del Gradiente Stocastica, applicate in particolare a problemi di **regressione**. Nella regressione, l'obiettivo è predire una quantità *continua*, come il prezzo di una casa, la temperatura di una città, o il valore di una misurazione di un sensore.

Spostiamo ora la nostra attenzione su un altro ramo fondamentale dell'apprendimento supervisionato: la **classificazione**.

## Nota

*Figura del testo originale (non riprodotta qui):* un'illustrazione che confronta un problema di regressione (predizione di un valore continuo su una retta) con un problema di classificazione (separazione di punti dati in due classi tramite un confine di decisione).

## 1 Dalla regressione alla classificazione

In un problema di **classificazione**, l'obiettivo *non* è predire un valore continuo, ma assegnare ogni ingresso  $x \in \mathbb{R}^d$  a una tra un insieme di **classi discrete**. Esempi tipici includono:

- Classificare le email come *spam* o *non spam*,
- Determinare se un'immagine medica mostra tessuto *sano* o un *tumore*,
- Riconoscere cifre scritte a mano (0–9),
- Predire se un cliente *acquisterà* un prodotto o *no*.

Nel caso più semplice (cioè la classificazione binaria), vogliamo predire se un'etichetta  $y$  appartiene a una tra due classi, che indicheremo con

$$y \in \{0, 1\}.$$

Questa impostazione somiglia molto alla regressione lineare: riceviamo un ingresso  $x$  e produciamo un'uscita  $y = f_{\Theta}(x)$ . Tuttavia, mentre la regressione produce in uscita un numero reale, la classificazione richiede una **probabilità** o una **decisione discreta**.

Per questo motivo, partiremo da un'estensione naturale della regressione lineare alla classificazione binaria. L'idea chiave è semplice:

## Nota

Calcoliamo una predizione lineare come nella regressione, e poi la passiamo attraverso una **funzione sigmoide** per ottenere una probabilità in  $[0, 1]$ .

Formalmente,

$$f_{\Theta}(x) := \sigma\left(\sum_{i=1}^n \Theta_i x_i\right) = \sigma(\Theta^T x),$$

dove

$$\sigma(t) = \frac{1}{1 + e^{-t}}$$

è la **funzione sigmoide**, che mappa qualsiasi numero reale in un valore tra 0 e 1. Interpretiamo  $f_{\Theta}(x)$  come la probabilità che  $x$  sia associato alla classe  $y = 1$ , cioè:

$$\sigma(\Theta^T x) = \mathbb{P}(y = 1 \mid x).$$

Abbiamo così trasformato un modello di regressione lineare in un **classificatore probabilistico**.

Questo modello è noto come **regressione logistica**, e rappresenta il mattone fondamentale dei moderni classificatori binari, incluse le reti neurali e i modelli di deep learning.

Prima di derivarne la procedura di training, esploriamo perché la sigmoide sia una buona scelta e in cosa la classificazione differisca dalla regressione in termini di modellazione e funzioni obiettivo.

#### Nota

*Figura del testo originale (non riprodotta qui):* il grafico della funzione sigmoide  $\sigma(t) = 1/(1 + e^{-t})$ , a forma di “S”, che tende a 0 per  $t \rightarrow -\infty$ , a 1 per  $t \rightarrow +\infty$ , e vale 0.5 in  $t = 0$ .

## 2 La funzione sigmoide e l’interpretazione probabilistica

Per passare dalla regressione alla classificazione, abbiamo bisogno di un meccanismo che trasformi un’uscita a valori reali in una probabilità. La **funzione sigmoide** svolge perfettamente questo ruolo, poiché mappa qualsiasi numero reale  $t \in \mathbb{R}$  nell’intervallo  $(0, 1)$ , rendendola ideale per modellare probabilità.

### 2.1 Perché la sigmoide?

Consideriamo un modello lineare (come quello visto nel capitolo precedente),

$$z = \Theta^T x.$$

Poiché  $z$  può essere negativo o positivo, non possiamo interpretarlo direttamente come una probabilità. La sigmoide, tuttavia, ha le seguenti proprietà desiderabili:

- **Codominio:**  $\sigma(t) \in (0, 1)$ , il che rende facile interpretarne l’uscita come una probabilità,
- **Monotonia:** punteggi lineari maggiori corrispondono a probabilità più alte,
- **Regolarità (smoothness):** continuamente differenziabile (necessaria per i metodi basati sul gradiente),

### 2.2 Interpretazione del modello logistico

Interpretiamo l’uscita del modello come

$$\sigma(\Theta^T x) = \mathbb{P}(y = 1 \mid x; \Theta).$$

La **regola di decisione** per la classificazione binaria diventa:

$$\hat{y} = \begin{cases} 1 & \text{se } \sigma(\Theta^T x) \geq 0.5, \\ 0 & \text{altrimenti.} \end{cases}$$

Questo crea un **confine di decisione** (*decision boundary*) dato da

$$\Theta^T x = 0,$$

che è un iperpiano, esattamente come nella regressione lineare, il che significa che la regressione logistica produce un **classificatore lineare**.

## 2.3 Interpretazione geometrica

Il segno della funzione lineare determina la classe predetta. La sigmoide semplicemente “ammorbidisce” questa decisione, fornendo una *probabilità* anziché un’assegnazione netta.

Intuitivamente:

- Se  $\Theta^T x \gg 0$ : alta confidenza nella classe 1
- Se  $\Theta^T x \ll 0$ : alta confidenza nella classe 0
- Se  $\Theta^T x \approx 0$ : predizione incerta (probabilità  $\approx 0.5$ )

## 2.4 Gradiente della sigmoide

Per allenare il modello tramite metodi basati sul gradiente, abbiamo bisogno della sua derivata:

$$\sigma'(t) = \sigma(t)(1 - \sigma(t)).$$

Questa forma elegante (un prodotto tra l’uscita e il suo complemento) spiega perché la sigmoide è estremamente comoda per l’ottimizzazione.

## 3 Binary Cross-Entropy Loss

Ora che modelliamo la probabilità che un campione appartenga alla classe 1 come

$$\hat{y} = \sigma(\Theta^T x), \quad \hat{y} \in (0, 1),$$

abbiamo bisogno di una funzione di perdita appropriata per misurare quanto bene il modello si adatti alle etichette vere.

Poiché la classificazione produce in uscita delle *probabilità*, la loss corretta si deriva dalla **massima verosimiglianza** (*maximum likelihood*) e corrisponde alla **binary cross-entropy** (chiamata anche logistic loss). Sia  $y \in \{0, 1\}$  l’etichetta vera. Interpretiamo il modello come se assegnasse la probabilità

$$\mathbb{P}(y = 1 \mid x; \Theta) = \hat{y}, \quad \mathbb{P}(y = 0 \mid x; \Theta) = 1 - \hat{y}.$$

Quindi, la verosimiglianza di osservare  $y$  è

$$\mathbb{P}(y \mid x; \Theta) = \hat{y}^y (1 - \hat{y})^{(1-y)}.$$

Massimizzare la verosimiglianza equivale a minimizzare la **log-verosimiglianza negativa**:

$$\ell(\Theta; x, y) = -(y \log \hat{y} + (1 - y) \log(1 - \hat{y})).$$

Sostituendo  $\hat{y} = f_{\Theta}(x) = \sigma(\Theta^T x)$ , la loss **binary cross-entropy (BCE)** diventa

$$\ell(\Theta; x, y) = -\left(y \log \sigma(\Theta^T x) + (1 - y) \log \left(1 - \sigma(\Theta^T x)\right)\right).$$

Per un dataset  $(X, Y) = \{(x^{(i)}, y^{(i)})\}_{i=1}^N$ , la loss empirica è quindi:

$$\mathcal{L}(\Theta; X, Y) = \frac{1}{N} \sum_{i=1}^N \ell(\Theta; x^{(i)}, y^{(i)}).$$

### 3.1 Perché la cross-entropy?

La cross-entropy è preferita rispetto all’MSE per la classificazione perché:

- Misura direttamente la **log-verosimiglianza** (fondata statisticamente),
- Penalizza più fortemente le predizioni sbagliate ma fatte con sicurezza,
- I suoi gradienti restano informativi anche quando  $\sigma(\Theta^T x)$  satura,
- Empiricamente, porta a un training **molto più veloce e affidabile**.

Si può pensare alla BCE come se dicesse al modello:

### Nota

“Assegna alta probabilità alle etichette vere e bassa probabilità a quelle sbagliate, e paga un prezzo alto se sei sicuro ma sbagli.”

## 3.2 Derivazione del gradiente

Alleniamo il modello di classificazione con una variante della Discesa del Gradiente Stocastica (SGD). Per farlo, dobbiamo prima derivare il gradiente necessario per l'ottimizzazione. Ricordiamo che:

$$\mathcal{L}(\Theta; X, Y) = \sum_{i=1}^N \ell(\Theta; x^{(i)}, y^{(i)}),$$

e quindi:

$$\nabla_{\Theta} \mathcal{L}(\Theta; X, Y) = \sum_{i=1}^N \nabla_{\Theta} \ell(\Theta; x^{(i)}, y^{(i)}),$$

per cui ci basta calcolare  $\nabla_{\Theta} \ell(\Theta; x^{(i)}, y^{(i)})$  per eseguire l'algoritmo SGD. Per farlo, poniamo:

$$\hat{y} = \sigma(z), \quad z = \Theta^T x, \quad \ell(\Theta; x, y) = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})].$$

Applichiamo la regola della catena:

$$\nabla_{\Theta} \ell = \frac{\partial \ell}{\partial z} \frac{\partial z}{\partial \Theta} = \frac{\partial \ell}{\partial z} x,$$

dato che  $\partial z / \partial \Theta = x$ . Differenziando  $\ell$  rispetto a  $z$ :

$$\frac{\partial \ell}{\partial z} = - \left[ y \cdot \frac{1}{\hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} + (1 - y) \cdot \frac{-1}{1 - \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z} \right] = - \frac{\partial \hat{y}}{\partial z} \left[ \frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} \right].$$

Usando che  $\sigma'(z) = \hat{y}(1 - \hat{y})$ , abbiamo  $\frac{\partial \hat{y}}{\partial z} = \hat{y}(1 - \hat{y})$ . Sostituendo:

$$\frac{\partial \ell}{\partial z} = -\hat{y}(1 - \hat{y}) \left[ \frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} \right].$$

Semplifichiamo la parentesi:

$$\frac{y}{\hat{y}} - \frac{1 - y}{1 - \hat{y}} = \frac{y(1 - \hat{y}) - (1 - y)\hat{y}}{\hat{y}(1 - \hat{y})} = \frac{y - y\hat{y} - \hat{y} + y\hat{y}}{\hat{y}(1 - \hat{y})} = \frac{y - \hat{y}}{\hat{y}(1 - \hat{y})}.$$

Quindi,

$$\frac{\partial \ell}{\partial z} = -\hat{y}(1 - \hat{y}) \cdot \frac{y - \hat{y}}{\hat{y}(1 - \hat{y})} = -(y - \hat{y}) = \hat{y} - y.$$

Sostituendo nella regola della catena:

$$\nabla_{\Theta} \ell(\Theta; x, y) = \frac{\partial \ell}{\partial z} x = (\hat{y} - y) x.$$

Pertanto, per qualsiasi batch  $\mathcal{M} \subseteq \{X, Y\}$ ,

$$\nabla_{\Theta} \mathcal{L}(\Theta; X, Y) = \sum_{i=1}^N (\sigma(\Theta^T x^{(i)}) - y^{(i)}) x^{(i)}.$$

Questo risultato è **molto comodo**:

- Ha la stessa forma della regressione lineare,
- Cambia solo il termine d'errore:  $\hat{y}^{(i)} - y^{(i)}$ ,
- La derivata della sigmoide si cancella magicamente durante la semplificazione.

Quindi, la regola di aggiornamento SGD / GD diventa

$$\Theta^{(k+1)} = \Theta^{(k)} - \eta_k (\sigma(\Theta^{(k),T} x^{(i)}) - y^{(i)}) x^{(i)}.$$

Questa semplicità è uno dei motivi per cui la regressione logistica rimane fondamentale.

### 3.3 Interpretazione del gradiente

- Se il modello predice una probabilità **troppo alta** per l'etichetta 1, spinge  $\Theta$  **verso il basso**.
- Se la probabilità è **troppo bassa**, spinge  $\Theta$  **verso l'alto**.
- L'entità dell'aggiornamento dipende dall'**errore di confidenza** e dall'**ingresso**.

La regressione logistica è quindi **un'estensione regolare, probabilistica e differenziabile della classificazione lineare**.

## 4 Valutazione dei modelli di classificazione

Nella regressione misuravamo le prestazioni usando metriche come l'Errore Quadratico Medio. Per la classificazione, il target non è più un numero reale, ma una **categoria**, quindi le metriche di valutazione devono riflettere la **qualità della decisione**, non la vicinanza numerica.

### 4.1 La matrice di confusione

Date etichette binarie  $y \in \{0, 1\}$  e predizioni  $\hat{y} \in \{0, 1\}$ , definiamo:

| Reale \ Predetto | 1 (Positivo)        | 0 (Negativo)        |
|------------------|---------------------|---------------------|
| 1 (Positivo)     | Vero Positivo (TP)  | Falso Negativo (FN) |
| 0 (Negativo)     | Falso Positivo (FP) | Vero Negativo (TN)  |

Questi quattro numeri formano la **matrice di confusione** e sono alla base di tutte le metriche seguenti.

### 4.2 Accuratezza (Accuracy)

$$\text{Acc} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}}$$

L'accuratezza risponde alla domanda: *“Quale frazione delle predizioni totali è corretta?”*

**Limite:** da sola, può essere fuorviante quando le classi sono molto sbilanciate (ad es., predire sempre “0” in un dataset composto per il 99% da zeri dà un'accuratezza del 99%, ma è inutile). *(Per ora ignoriamo la gestione dello sbilanciamento delle classi, ma questo spiega perché l'accuratezza spesso non basta.)*

### 4.3 Precision e Recall

Precision:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}$$

*“Dei campioni predetti come positivi, quanti sono effettivamente positivi?”*

Recall (detta anche True Positive Rate):

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}$$

*“Di tutti i campioni positivi, quanti ne abbiamo correttamente individuati?”*

Applicazioni diverse enfatizzano metriche diverse:

- Rilevamento spam: **alta precision** (evitare di marcare email vere come spam)
- Screening di malattie: **alta recall** (individuare quanti più pazienti malati possibile)

## 4.4 F1 Score

La **media armonica** di precision e recall:

$$F_1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

Utile quando serve un singolo numero che riassume sia precision che recall.

## 4.5 Esercizio di implementazione: valutare la regressione logistica

Dopo aver allenato il modello di regressione logistica tramite SGD:

1. Calcola  $\hat{y} = \sigma(\Theta^T x)$
2. Converti in predizioni  $\hat{y} \in \{0, 1\}$  usando una soglia (si parte con 0.5)
3. Calcola:
  - Matrice di confusione
  - Accuratezza
  - Precision, Recall, F1

## 5 Esempio: regressione logistica su un dataset reale

Per concludere questo capitolo, alleniamo un modello di regressione logistica **da zero** (solo numpy) su un dataset di classificazione reale da Kaggle:

**Pima Indians Diabetes Database:** dataset Kaggle “*uciml/pima-indians-diabetes-database*” (link)

Questo dataset contiene misurazioni mediche di pazienti donne e un’etichetta binaria che indica se è stato diagnosticato il diabete. Similmente a quanto fatto in precedenza per la regressione, faremo:

- Caricamento del dataset
- Standardizzazione delle feature
- Training della regressione logistica con SGD
- Valutazione dell’accuratezza

### 5.1 Caricamento e preparazione dei dati

```
import numpy as np
import pandas as pd

# Scarica "diabetes.csv" da Kaggle e mettilo nella cartella del notebook
df = pd.read_csv("data/diabetes.csv")

X = df.drop("Outcome", axis=1).values # feature
y = df["Outcome"].values.reshape(-1,1) # etichette

# Standardizza le feature
X_mean, X_std = X.mean(axis=0), X.std(axis=0)
X = (X - X_mean) / X_std

# Aggiunge il termine di bias
X = np.hstack([np.ones((X.shape[0],1)), X])
N, d = X.shape
```

### 5.2 Regressione logistica (training con SGD)

```

def sigmoid(z):
    return 1/(1+np.exp(-z))

def l(theta, X, y):
    p = sigmoid(X @ theta)
    return -(y*np.log(p) + (1-y)*np.log(1-p)).mean()

def grad_l(theta, X, y):
    return (X.T @ (sigmoid(X @ theta) - y)) / len(y)

def accuracy(theta, X, y):
    preds = sigmoid(X @ theta) >= 0.5
    return (preds == y).mean()

def train_sgd(l, grad_l, X, y, lr=0.1, epochs=200, batch_size=32):
    N, d = X.shape
    theta = np.zeros((d, 1))
    losses, accs = [], []

    for epoch in range(epochs):
        # Mescola i dati
        idx = np.arange(len(X))
        np.random.shuffle(idx)

        X = X[idx, :]
        y = y[idx]

        for batch in range(0, N, batch_size):
            grad = grad_l(theta,
                           X[batch:batch + batch_size, :],
                           y[batch:batch + batch_size, :])
            theta -= lr * grad

        # Registra le metriche sull'intero dataset dopo l'epoca
        losses.append(l(theta, X, y))
        accs.append(accuracy(theta, X, y))

    return theta, np.array(losses), np.array(accs)

theta_sgd, loss_sgd, acc_sgd = train_sgd(l, grad_l, X, y,
                                          lr=1e-3, epochs=200, batch_size=32)
print(f"Accuratezza SGD: {acc_sgd[-1]*100:.2f}%")

```

Accuratezza SGD: 76.43%

### 5.3 Training con Adam

```

def train_adam(l, grad_l, X, y, lr=1e-3, epochs=200, batch_size=32,
               beta1=0.9, beta2=0.999, eps=1e-8,
               ):
    N, d = X.shape
    theta = np.zeros((d, 1))
    m = np.zeros_like(theta)
    v = np.zeros_like(theta)
    t = 0

    losses, accs = [], []

    for epoch in range(epochs):
        # Mescola i dati
        idx = np.arange(len(X))
        np.random.shuffle(idx)

```

```

X = X[idx, :]
y = y[idx]
for batch in range(0, N, batch_size):
    g = grad_l(theta, X[batch:batch + batch_size], y[batch:batch + batch_size])

    t += 1
    m = beta1 * m + (1 - beta1) * g
    v = beta2 * v + (1 - beta2) * (g * g)
    m_hat = m / (1 - beta1 ** t)
    v_hat = v / (1 - beta2 ** t)

    theta -= lr * (m_hat / (np.sqrt(v_hat) + eps))

# Registra le metriche sull'intero dataset dopo l'epoca
losses.append(l(theta, X, y))
accs.append(accuracy(theta, X, y))

return theta, np.array(losses), np.array(accs)

theta_adam, loss_adam, acc_adam = train_adam(1, grad_l, X, y,
                                             lr=1e-3, epochs=200, batch_size=32)
print(f"Accuratezza Adam: {acc_adam[-1]*100:.2f}%")

```

Accuratezza Adam: 78.26%

```

import matplotlib.pyplot as plt

# Grafico della loss
plt.figure()
plt.plot(loss_sgd, label="SGD - Loss")
plt.plot(loss_adam, label="Adam - Loss")
plt.xlabel("Epoca")
plt.ylabel("Binary Cross-Entropy (Loss)")
plt.title("Regressione Logistica: Loss per epoca")
plt.grid(alpha=0.3)
plt.legend()
plt.show()

# Grafico dell'accuratezza
plt.figure()
plt.plot(acc_sgd, label="SGD - Accuratezza")
plt.plot(acc_adam, label="Adam - Accuratezza")
plt.xlabel("Epoca")
plt.ylabel("Accuratezza")
plt.title("Regressione Logistica: Accuratezza per epoca")
plt.grid(alpha=0.3)
plt.legend()
plt.show()

```

### Nota

*Figure del testo originale (non riprodotte qui):* due grafici che mostrano rispettivamente l'andamento della loss (BCE) e dell'accuratezza per SGD e Adam lungo le epoche; entrambi gli ottimizzatori convergono, con Adam generalmente più rapido e leggermente più accurato.

## 6 Dalla regressione logistica a una semplice rete neurale

Fino a questo punto, i nostri modelli sono stati **lineari**: sia per la regressione che per la classificazione abbiamo sempre calcolato un unico punteggio lineare della forma

$$z = \Theta^T x, \quad \hat{y} = \sigma(z),$$



per poi allenare i parametri minimizzando una loss appropriata. Questo è il nucleo della **regressione logistica**, e come abbiamo visto funziona già sorprendentemente bene. Tuttavia, un'unica trasformazione lineare può separare solo dati linearmente separabili. In altre parole, la regressione logistica può apprendere solo **confini di decisione lineari**. Ma cosa succede se la vera relazione tra ingresso e uscita è più complessa? Ad esempio, cosa succede se le classi si trovano in regioni *curve* o *intrecciate* dello spazio? Questo motiva il prossimo passo: le **reti neurali**.

## 6.1 Aggiunta di un layer nascosto

L'idea alla base delle reti neurali è molto semplice:

### Nota

Invece di mappare direttamente le feature di ingresso in un'uscita, prima trasformiamo gli ingressi in una rappresentazione più ricca, e poi facciamo le predizioni basandoci su quella rappresentazione.

Facciamo questo inserendo un **layer nascosto** (*hidden layer*). Concretamente, prendiamo il nostro ingresso  $x$ , applichiamo una **mappa lineare**, poi una **funzione di attivazione non lineare**, e usiamo le feature risultanti per fare la predizione finale. Matematicamente, invece di

$$\hat{y} = \sigma(\Theta^T x),$$

calcoliamo

$$\begin{aligned} z_1 &= W_1 x + b_1, \\ a_1 &= \text{ReLU}(z_1), \\ z_2 &= W_2 a_1 + b_2, \\ \hat{y} &= \sigma(z_2), \end{aligned}$$

dove:

- $W_1, b_1$  sono i parametri del **layer nascosto**,
- $W_2, b_2$  sono i parametri del **layer di uscita (logistico)**,
- $\text{ReLU}(t) = \max(0, t)$  introduce la **non linearità**.

Questo modello può ora apprendere **confini di decisione non lineari**, perché la trasformazione ReLU può piegare e rimodellare lo spazio di ingresso prima che avvenga la classificazione.

Si noti quanto questo sia intuitivo: il layer ReLU sta apprendendo automaticamente nuove feature, invece che essere noi a costruirle manualmente.

## 6.2 Allenare la rete: la regola della catena in azione

Anche se il modello è più complesso, il principio del training resta invariato:

### Nota

Calcoliamo la loss, e poi aggiustiamo i parametri per ridurla.

La differenza chiave è che ora abbiamo più parametri e una composizione di funzioni. Fortunatamente, le regole di differenziazione si estendono perfettamente: applichiamo semplicemente la **regola della catena** layer per layer. Questo processo, noto come **backpropagation**, non è altro che l'applicazione ripetuta di:

$$\frac{d}{dx} f(g(x)) = f'(g(x)) \cdot g'(x).$$

Anche la non linearità ReLU ha una derivata molto semplice:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

Durante la backpropagation, questo significa che i gradienti possono “fluire” solo attraverso i neuroni che erano attivi (cioè che avevano un ingresso positivo). I neuroni con ingresso negativo semplicemente non contribuiscono.

Questa semplice idea ci permette già di costruire un approssimatore universale di funzioni: una rete con un solo layer nascosto può teoricamente approssimare **qualsiasi funzione continua**, a patto di usare un numero sufficiente di unità nascoste.

Naturalmente, in pratica, le reti più profonde tendono ad apprendere in modo più efficace, ma questa è una storia per i corsi successivi.

### 6.3 Implementazione da zero

Di seguito è riportata un’implementazione compatta e completamente vettorizzata di una rete neurale con un layer nascosto per la classificazione binaria.

Questo codice:

- Calcola il passo forward,
- Calcola tutti i gradienti manualmente tramite la regola della catena,
- Applica aggiornamenti SGD,
- Registra loss e accuratezza durante il training.

Si riconoscerà la struttura familiare della regressione logistica, semplicemente con un layer in più.

```
import numpy as np

def sigmoid(z):
    return np.where(z >= 0, 1/(1+np.exp(-z)), np.exp(z)/(1+np.exp(z)))

def relu(z):
    return np.maximum(0, z)

def relu_grad(z):
    g = np.zeros_like(z)
    g[z > 0] = 1.0
    return g

def bce_loss(y_hat, y, eps=1e-10):
    y_hat = np.clip(y_hat, eps, 1 - eps)
    return -(y*np.log(y_hat) + (1-y)*np.log(1 - y_hat)).mean()

def train_mlp_adam(X, y, hidden=16, lr=1e-2, epochs=200, batch_size=32):
    N, d = X.shape

    W1 = np.random.normal(0, np.sqrt(2/d), size=(d, hidden))
    b1 = np.zeros((1, hidden))
    W2 = np.random.normal(0, np.sqrt(2/hidden), size=(hidden, 1))
    b2 = np.zeros((1, 1))

    losses, accs = [], []

    for _ in range(epochs):
        # Mescola i dati
        idx = np.arange(len(X))
        np.random.shuffle(idx)

        X = X[idx, :]
        y = y[idx]

        for batch in range(0, N, batch_size):
            Xb, yb = X[batch:batch+batch_size], y[batch:batch+batch_size]

            # forward
            z1 = Xb @ W1 + b1
```

```

        a1 = relu(z1)
        z2 = a1 @ W2 + b2
        y_hat = sigmoid(z2)

        # backward
        delta2 = (y_hat - yb) / len(yb)
        dW2 = a1.T @ delta2
        db2 = delta2.sum(axis=0, keepdims=True)

        delta1 = (delta2 @ W2.T) * relu_grad(z1)
        dW1 = Xb.T @ delta1
        db1 = delta1.sum(axis=0, keepdims=True)

        # aggiornamento
        W1 -= lr * dW1
        b1 -= lr * db1
        W2 -= lr * dW2
        b2 -= lr * db2

        # tracciamento
        z1 = X @ W1 + b1
        a1 = relu(z1)
        z2 = a1 @ W2 + b2
        yhat_full = sigmoid(z2)

        losses.append(bce_loss(yhat_full, y))
        preds = (yhat_full >= 0.5).astype(int)
        accs.append((preds == y).mean())

    return (W1, b1, W2, b2), np.array(losses), np.array(accs)

theta_nn_sgd, loss_nn_sgd, acc_nn_sgd = train_mlp_adam(
    X, y,
    hidden=16,
    lr=1e-3,
    epochs=200,
    batch_size=32,
)
print(f"Accuratezza finale della rete neurale: {acc_nn_sgd[-1]*100:.2f}%")

```

```

Accuratezza finale della rete neurale: 75.78%

```

```

# Grafico delle curve di loss
plt.figure(figsize=(6,4))
plt.plot(loss_sgd, label="Regressione Logistica (SGD) - Loss", alpha=0.8)
plt.plot(loss_adam, label="Regressione Logistica (Adam) - Loss", alpha=0.8)
plt.plot(loss_nn_sgd, label="Rete Neurale (SGD) - Loss", alpha=0.8)
plt.xlabel("Epoca")
plt.ylabel("Binary Cross-Entropy Loss")
plt.title("Confronto delle Loss")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

# Grafico delle curve di accuratezza
plt.figure(figsize=(6,4))
plt.plot(acc_sgd, label="Regressione Logistica (SGD) - Accuratezza", alpha=0.8)
plt.plot(acc_adam, label="Regressione Logistica (Adam) - Accuratezza", alpha=0.8)
plt.plot(acc_nn_sgd, label="Rete Neurale (SGD) - Accuratezza", alpha=0.8)
plt.xlabel("Epoca")
plt.ylabel("Accuratezza")
plt.title("Confronto delle Accuratezze")
plt.legend()
plt.grid(alpha=0.3)

```

```
plt.show()
```

### Nota

*Figure del testo originale (non riprodotte qui):* due grafici di confronto tra regressione logistica (SGD e Adam) e rete neurale (SGD) sulla loss e sull'accuratezza lungo le epoche.

## 6.4 Osservazioni finali

Ciò che abbiamo costruito qui è la rete neurale più semplice possibile. Eppure, concettualmente, contiene già **tutti gli ingredienti essenziali del deep learning**:

- Impilare trasformazioni lineari,
- Inserire non linearità,
- Applicare la regola della catena all'indietro attraverso il modello,
- Aggiornare i parametri tramite ottimizzazione basata sul gradiente.

Tutto ciò di cui si è forse sentito parlare, come le reti convoluzionali, i transformer, i modelli di diffusione, gli LLM, è costruito sopra questa stessa base.

Passando dalla regressione logistica a una rete con un layer nascosto, iniziamo a vedere come le macchine possano **apprendere pattern potenti e non lineari** dai dati, invece di affidarsi a feature costruite manualmente. Da qui, continueremo ad aumentare espressività, profondità e scalabilità, sempre guidati dagli stessi principi matematici fondamentali introdotti in questo corso.

---

*Traduzione italiana del materiale del corso “Statistical and Mathematical Methods for Machine Learning”  
(pagina originale in inglese).*

*Materiale originale di Davide Evangelista — © 2023.*